Phase 4: สร้าง SSIS Control Flow

Created	@February 22, 2026 5:39 PM
Tags	

Phase นี้เป็นหัวใจหลักของโปรเจคเลย เราจะเริ่มที่ ETL_10k.dtsx ก่อน

ภาพรวม Task ทั้งหมดที่จะสร้าง

```
1. Script Task: Start Timer
        ↓
2. Data Flow Task: Extract
        ↓
3. Script Task: Log Extract Time
        ↓
4. Data Flow Task: Transform
        ↓
5. Script Task: Log Transform Time
        ↓
6. Data Flow Task: Load
        ↓
7. Script Task: Log Load + Write CSV
```

Task 1 : Script Task: Start Timer

> Script Task คือ Task ที่เปิดให้เราเขียนโค้ด C# เองได้ ใช้ในกรณีที่สิ่งที่เราต้องการทำไม่มี Task สำเร็จรูปใน SSIS รองรับ เช่น การจับเวลา การคำนวณ หรือการเขียนไฟล์ CSV ซึ่งทั้งหมดนี้ SSIS ไม่มี Task สำเร็จรูปให้ใช้
> ในโปรเจคนี้ใช้ Script Task ทั้งหมด 4 จุด คือ Task 1 บันทึกเวลาเริ่มต้น Task 3 คำนวณเวลาที่ใช้ใน Extract Task 5 คำนวณเวลาที่ใช้ใน Transform และ Task 7 คำนวณเวลาที่ใช้ใน Load แล้วเขียนผลทั้งหมดลงไฟล์ CSV

เริ่มจาก Task แรกก่อนเลย:

1. Double-click **ETL_10k.dtsx** เพื่อเปิด
2. ดูที่ **SSIS Toolbox** ด้านซ้าย → หา **Script Task**
3. **ลาก Script Task** ลงมาวางใน Control Flow Design Surface
4. คลิกขวา ที่ Script Task → เปลี่ยนชื่อเป็น `Start Timer`
5. Double-click แล้ว ตั้งค่าดังนี้:
   - **ScriptLanguage:** Microsoft Visual C# 2026
   - **ReadWriteVariables:** `User::ExtractStart`

   > User:: คือ prefix ที่ SSIS ใส่ให้อัตโนมัติ เพื่อบอกว่า Variable ตัวนี้เป็นตัวที่เราสร้างเองใน STEP 3 ไม่ใช่ Variable ที่ SSIS สร้างไว้ให้ในระบบ การใส่ User:: ไว้ข้างหน้าทำให้ SSIS รู้ว่าต้องไปหา Variable ตัวนี้จากที่ที่เราสร้างไว้

   > ReadWriteVariables คือการบอก Script Task ว่าจะขอสิทธิ์เข้าถึง Variable ตัวไหนบ้าง SSIS บังคับให้ต้องระบุไว้ก่อนเสมอ ถ้าไม่ระบุแล้ว Script พยายามเขียนค่าลง Variable ตัวนั้น SSIS จะ Error ทันที

6. กด **Edit Script** → หาโค้ด `public void Main()`

   วางโค้ดนี้:

```
public void Main()
{
    Dts.Variables["User::ExtractStart"].Value = DateTim
e.Now;
    /*
        บรรทัดนี้บันทึกเวลาปัจจุบัน ณ ตอนที่ Task นี้ทำงานลงใน Vari
able ชื่อ ExtractStart
        ส่วน DateTime.Now คือคำสั่งที่ดึงเวลาปัจจุบันของเครื่อง
```

```
        และ .Value คือการบอกว่าให้เอาค่านี้ไปเก็บไว้ใน Variable
    */

     Dts.TaskResult = (int)ScriptResults.Success;
     /*
        บรรทัดนี้คือการบอก SSIS ว่า Task นี้ทำงานเสร็จแล้วและสำเร็จ
        ถ้าไม่ใส่ SSIS จะไม่รู้ว่า Task ทำงานเสร็จหรือยัง และจะไม่ยอม
เดินหน้าไปทำ Task ถัดไป
    */
}
```

`Ctrl+S`

7. Build → ปิด Script Editor → กด **OK**

Task 2 : Data Flow Task - Extract (Extract from SQL Server)

Data Flow Task คือ Task ที่ใช้สำหรับเคลื่อนย้ายข้อมูลข้างในจะมีพื้นที่แยกสำหรับกำหนดว่าข้อมูลจะไหลจากที่ไหน ผ่านอะไรบ้าง และไปลงที่ไหน ต่างจาก Script Task ตรงที่ Script Task ใช้เขียนโค้ดเพื่อทำงานทั่วไป แต่ Data Flow Task ออกแบบมาเพื่อจัดการข้อมูลโดยเฉพาะ
ในโปรเจคนี้ต้องการจับเวลาแยกกันเป็น 3 Phase ดังนั้นเราต้องแยกเป็น Task ละ Phase ทำให้จับเวลาได้ว่า Extract ใช้เวลากี่วินาที Transform ใช้เวลากี่วินาที และ Load ใช้เวลากี่วินาที

1. ลาก **Data Flow Task** จาก SSIS Toolbox ลงใน Control Flow
2. เปลี่ยนชื่อเป็น `Extract from SQL Server`
3. เชื่อมลูกศร (**Precedence Constraint**) โดยคลิกที่ `Start Timer` แล้วลากลูกศรไปที่ `Extract from SQL Server`
4. Double-click เข้าไปข้างใน Data Flow Task
5. ลาก **ADO NET Source** จาก SSIS Toolbox ตรง other sources ลงมาใน Data Flow

> ADO NET Source คือ Component ที่ทำหน้าที่ดึงข้อมูลจากฐานข้อมูลเข้ามาใน Data Flow ที่ใช้ ADO NET Source เพราะเราสร้าง Connection Manager ของ

> SQL Server ไว้แบบ ADO.NET ใน STEP 4.1 ใน phase 3 แล้ว ADO NET Source จึงสามารถใช้ Connection Manager ตัวนั้นเชื่อมต่อและดึงข้อมูลจากตาราง fraud_10k ได้

6. Double-click ที่ **ADO NET Source** → ตั้งค่า:

Field	ค่าที่กรอก
ADO NET Source	DESKTOP-2FDM07U_SQLEXPRESS.frauddb.sk
Data Access Mode	`Table or view`
Table	`[dbo].[fraud_10k]`

กด **Preview** เพื่อตรวจสอบว่าดึงข้อมูลได้ → กด **OK**

Task 3 : Script Task - Log Extract Time

1. กลับไปที่ **Control Flow** (คลิก tab Control Flow ด้านบน)
2. ลาก **Script Task** ลงมาใน Control Flow
3. เปลี่ยนชื่อเป็น `Log Extract Time`
4. เชื่อมลูกศรจาก **Extract from SQL Server** → **Log Extract Time**
5. Double-click → ตั้งค่า:
   - **ReadWriteVariables:** `User::ExtractStart,User::ExtractSec,User::TransformStart`

   > Task 3 ต้องทำงาน 3 อย่างพร้อมกัน คือ อ่านค่าจาก ExtractStart เพื่อรู้ว่า Extract เริ่มเมื่อกี่โมง เขียนผลลัพธ์ที่คำนวณได้ลง ExtractSec ว่าใช้เวลากี่วินาที และเริ่มจับเวลา Transform ทันทีโดยบันทึกเวลาปัจจุบันลง TransformStart

6. กด **Edit Script** → วางโค้ดนี้ที่ `public void Main()`:

```
public void Main()
{
    var start = (DateTime)Dts.Variables["User::ExtractStar
t"].Value;
    var elapsed = DateTime.Now - start;  // ได้ผลลัพธ์เป็นระยะเว
ลาที่ผ่านไป
    Dts.Variables["User::ExtractSec"].Value = elapsed.Total
Seconds;
    // TotalSeconds จะแปลงทุกอย่างให้เหลือแค่จำนวนวินาที
```

```
    // elapsed อย่างเดียวจะได้ผลลัพธ์เป็นรูปแบบเวลาเต็มๆ
    Dts.Variables["User::TransformStart"].Value = DateTime.
Now;
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

`Ctrl+S`

7. Build → ปิด Script Editor → กด **OK**

> 💡 Task นี้คำนวณว่า Extract ใช้เวลากี่วินาที แล้วเริ่มจับเวลา Transform ต่อทันที ถ้ารอบันทึกใน Task 4 เวลาที่ได้จะไม่แม่นยำ Task 3 และ Task 4 SSIS ต้องใช้เวลาเตรียมตัวก่อนที่ Data Flow Task จะเริ่มทำงานจริง ถ้าบันทึกเวลาไว้ตรงท้าย Task 3 พอดีก่อนที่จะส่งต่อให้ Task 4 เวลาที่จับได้จะแม่นยำกว่า

Task 4 : Data Flow Task: Transform

1. ลาก **Data Flow Task** ลงใน Control Flow
2. เปลี่ยนชื่อเป็น `Transform`
3. เชื่อมลูกศรจาก **Log Extract Time** → **Transform**
4. Double-click เข้าไปข้างใน Data Flow Task
5. ลาก **ADO NET Source** ลงมา → Double-click → ตั้งค่าเลือก Table = `fraud_10k, fraud_100k,` เหมือน Task 2
6. กด **OK**

Task 4.1 : ตั้งค่า Script Component (เพิ่ม Script Component (Transformation))

ยังอยู่ใน Data Flow ของ Transform:

1. ลาก **Script Component** ลงมาต่อ → เลือก Type = **Transformation** → กด OK
2. เชื่อมลูกศรจาก **ADO NET Source** → จะเห็นลูกศรสีน้ำเงินโผล่ออกมา → ลากลูกศรไปวางที่ **Script Componen**

> ทำไม Task 4 ต้องดึงข้อมูลจาก SQL Server อีกครั้ง ทั้งที่ Task 2 ก็ดึงมาแล้ว? เพราะข้อมูลที่ดึงมาใน Task 2 ลอยอยู่ใน Memory ชั่วคราวและหายไปแล้วตั้งแต่ Task 2 จบลง แต่ละ Data Flow Task จะดึงข้อมูลของตัวเองขึ้นมาใหม่ทุกครั้ง Task 2

> ดึงมาเพื่อนับเวลา Extract อย่างเดียว ส่วน Task 4 ดึงมาเพื่อ Transform และ Load เข้า MySQL

Task 4.2 : ตั้งค่า Script Component

Double-click ที่ **Script Component** จะเห็นหน้าต่าง Script Transformation Editor การตั้งค่า Script Component จะมีอยู่ 3 ขั้นตอนหลัก ดังนี้:

Task 4.2.1 : ตั้งค่า Tab Input Columns (ซ้ายมือ)

- ติ๊กถูกทุก column ที่ต้องการ **ยกเว้น** `column1` เพราะ `column1` คือ **Unnamed: 0** จาก CSV ซึ่งเป็นแค่ index ลำดับแถว ไม่มีความหมายทางธุรกิจเลย เราไม่ต้องการนำไปใช้ใน Transform หรือเก็บลง MySQL

Task 4.2.2 : เพิ่ม Output Columns ใน Script Component ไปที่ Tab Inputs and Outputs (ซ้ายมือ)

> Output Columns คือ Column ใหม่ที่ Script Component สร้างขึ้นมาเพื่อเก็บผลลัพธ์หลังจาก Transform แล้ว SSIS จะไม่ยอมให้เขียนทับ Column ที่รับมาจาก Input โดยตรงดังนั้นเราจึงต้องสร้าง Column ใหม่ขึ้นมา

- ขยาย **Output 0** → คลิก **Output Columns** → กด **Add Column**
  - กด **Add Column** → แก้ชื่อเป็น `cc_num_hash` → DataType = `string [DT_STR]` → Length = `64`
  - กด **Add Column** → แก้ชื่อเป็น `first_hash` → DataType = `string [DT_STR]` → Length = `64`
  - กด **Add Column** → แก้ชื่อเป็น `last_hash` → DataType = `string [DT_STR]` → Length = `64`
  - กด **Add Column** → แก้ชื่อเป็น `dob_hash` → DataType = `string [DT_STR]` → Length = `64`
  - กด **Add Column** → แก้ชื่อเป็น `trans_dt_clean` → DataType = `database timestamp [DT_DBTIMESTAMP]` → Length ไม่ต้องใส่
- OK

Task 4.2.3 : Edit Script ใน Script Component ใน Tab Scripts (ซ้ายมือ):

1. คลิก Tab **Script** → กด **Edit Script**
2. วางโค้ดนี้ข้างบน `public override void Input0_ProcessInputRow(Input0Buffer Row)` :

```
private string HashSHA256(string input)
{
    using (var sha = SHA256.Create())
    {
        byte[] bytes = sha.ComputeHash(Encoding.UTF8.Get
Bytes(input ?? ""));
        return BitConverter.ToString(bytes).Replace("-",
"").ToLower();
    }
}
```

3. วางโค้ดนี้ใน `Input0_ProcessInputRow`:

```
    Row.ccnumhash = HashSHA256(Row.ccnum.ToString());
    Row.firsthash = HashSHA256(Row.first);
    Row.lasthash = HashSHA256(Row.last);
    Row.dobhash = HashSHA256(Row.dob.ToString());
    Row.transdtclean = Row.transdatetranstime;
```

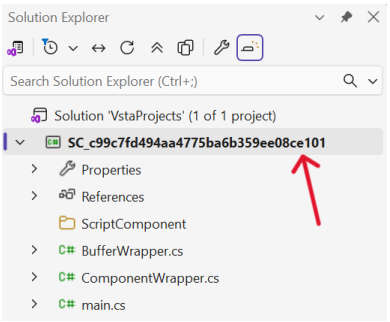
4. เพิ่ม using ด้านบนที่ `#region Namespaces`:

```
using System.Security.Cryptography;
using System.Text;
```

5. กด **Build** → **Build SC_...**
6. ปิด Script Editor → กด **OK**

ถ้า Build ไม่สำเร็จ ลองเปลี่ยนตามวิธีนี้ดู:

1. ไปที่ **Build** → **Configuration Manager** → ไปที่ Active solution platform → New → เปลี่ยน Platform เป็น **x64** → OK → Close
2. ไปที่ Solution Explorer ด้านขวา → จะเห็นตัวหนาที่แสดง Assembly name → คลิกขวา → **Properties**



3. เปลี่ยน **Target Framework** เป็น **.NET Framework 4.7.2**
4. กด **Build ใหม่อีกครั้ง** → **Build SC_...**
5. ปิด Script Editor → กด **OK**

Task 4.3 : เพิ่ม Conditional Split (กรอง amt <= 0)

1. ลาก **Conditional Split** ลงใน Data Flow

> Conditional Split คือ Component ที่ทำหน้าที่แยกแถวข้อมูลออกเป็น 2 ทางตามเงื่อนไขที่กำหนด ในโปรเจคนี้กำหนดเงื่อนไขว่าถ้า amt <= 0 หรือ amt เป็น NULL ให้แยกออกไปเป็น Invalid เนื่องจากธุรกรรมที่มียอดเงิน 0 หรือติดลบไม่มีความหมายในเชิงการวิเคราะห์การฉ้อโกง เพราะการฉ้อโกงจะต้องมีการใช้เงินจริงๆ ถ้าเอาแถวพวกนี้เข้าไปด้วยจะทำให้ผลการวิเคราะห์คลาดเคลื่อน ส่วนแถวที่ผ่านเงื่อนไขจะไหลต่อไปเป็น Valid

2. เชื่อมลูกศรจาก **Script Component** → **Conditional Split**
3. Double-click → ตั้งค่า:
   - Output Name: `Invalid`
   - Condition: `amt <= 0 || ISNULL(amt)`
   - Default Output: `Valid`

4. กด **OK**

> 💡 **Conditional Split** ทำหน้าที่เหมือน if-else คือแถวที่ `amt <= 0` จะถูกแยกออกไปเป็น Invalid ส่วนแถวปกติจะผ่านไปเป็น Valid

---

Task 4.4 : เพิ่ม ODBC Destination (Load เข้า MySQL)

ODBC Destination คือ Component ที่ทำหน้าที่รับข้อมูลปลายทางและส่งเข้าฐานข้อมูล ในที่นี้ใช้ส่งข้อมูลเข้า MySQL โดยใช้ Connection Manager ที่สร้างไว้ใน STEP 4.2 ของ Phase 3 ที่ต้องเชื่อมจาก Valid เพราะเราต้องการเฉพาะแถวที่ผ่านการกรองแล้วเท่านั้น ส่วน Invalid คือแถวที่ถูกคัดออกซึ่งไม่ต้องการเก็บไว้

1. ลาก **ODBC Destination** จาก Toolbox ลงใน Data Flow
2. เชื่อมลูกศรจาก **Conditional Split** → **ODBC Destination** โดยเลือก Output = `Valid`
3. Double-click ที่ ODBC Destination → ตั้งค่า:

Field	ค่าที่กรอก
ODBC Connection Manager	`MySQL_FraudDL.root`
Data access mode:	Table Name - batch
Name of the table or the view	`fraud_clean_ssis`
เลือก	Table Name - row by row

Table Name - row by row
ส่งข้อมูลทีละ 1 แถว เหมือนขนของทีละชิ้น ช้ากว่า แต่ถ้า Error จะรู้ได้ทันทีว่าแถวไหนมีปัญหา
Table Name - batch
ส่งข้อมูลทีละหลายแถวพร้อมกัน เหมือนขนของทีละกล่อง เร็วกว่ามาก แต่ถ้า Error จะไม่รู้ว่าแถวไหนผิด

4. คลิก Tab **Mappings** → Map column ให้ตรงกัน:

> Mapping คือการบอก SSIS ว่า Column ที่ไหลมาจาก Data Flow ตัวไหนควรเอาไปเก็บใน Column ไหนของตาราง MySQL ต้องทำเพราะชื่อ Column ใน Data Flow กับชื่อ Column ใน MySQL อาจไม่ตรงกันทุกตัว เช่น trans_dt_clean ใน

> Data Flow ต้องจับคู่กับ trans_datetime ใน MySQL ถ้าไม่ทำ Mapping SSIS จะไม่รู้ว่าต้องเอาข้อมูลไปใส่ Column ไหน

Input Column	Destination Column
cc_num_hash	cc_num_hash
first_hash	first_hash
last_hash	last_hash
dob_hash	dob_hash
trans_dt_clean	trans_datetime
merchant	merchant
category	category
amt	amt
gender	gender
city	city
state	state
city_pop	city_pop
job	job
trans_num	trans_num
is_fraud	is_fraud

กด **OK**

Task 5 : Script Task: Log Transform Time

กลับไปที่ **Control Flow**

1. ลาก **Script Task** ลงใน Control Flow
2. เปลี่ยนชื่อเป็น `Log Transform Time`
3. เชื่อมลูกศรจาก **Transform** → **Log Transform Time**
4. Double-click → ตั้งค่า:
   - **ReadWriteVariables:** `User::TransformStart,User::TransformSec,User::LoadStart`

     > อ่านค่าจาก TransformStart แล้วเขียนผลลงใน TransformSec และเริ่มจับเวลา Load
5. กด **Edit Script** → วางโค้ดนี้ใน `Main()`:

```
public void Main()
{
        // TODO: Add your code here
    var start = (DateTime)Dts.Variables["User::TransformSta
rt"].Value;
    var elapsed = DateTime.Now - start;
    Dts.Variables["User::TransformSec"].Value = elapsed.Tot
alSeconds;
    Dts.Variables["User::LoadStart"].Value = DateTime.Now;
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

Save → **Build** → ปิด Script Editor → กด **OK**

> Task 5 ทำงานเหมือน Task 3 ทุกอย่างแต่เปลี่ยนจากการคำนวณเวลา Extract มาเป็นการคำนวณเวลา Transform แทน และบันทึกเวลาเริ่ม Load ไว้ทันทีก่อนส่งต่อให้ Task 6 เพื่อให้เวลาที่จับได้แม่นยำที่สุด

Task 6 : Data Flow Task: Load เข้า MySQL

1. ลาก **Data Flow Task** ลงใน Control Flow
2. เปลี่ยนชื่อเป็น `Load to MySQL`
3. เชื่อมลูกศรจาก **Log Transform Time** → **Load to MySQL**

> 💡 **สังเกต:** Task 6 นี้ไม่ต้องทำอะไรเพิ่มแล้ว เพราะเราได้ Load ข้อมูลเข้า MySQL ไปแล้วใน Task 4 (ODBC Destination) ดังนั้น Task 6 เราจะทำแค่นี้ในโปรเจคนี้จะข้ามไปทำ **Task 7 — Log Load + Write CSV** เลย

> **ถ้าไม่ต้องทำอะไร แล้วทำไมต้องสร้าง Task นี้ขึ้นมาด้วย?**
> เพราะเราต้องการวัดเวลาครบทั้ง 3 Phase คือ Extract Transform และ Load ถ้าไม่มี Task 6
> Task 5 จะบันทึกเวลาเริ่ม Load ลง LoadStart แต่ไม่มี Task ที่เป็นจุดสิ้นสุดของ Load ให้ Task 7 ใช้คำนวณว่า Load ใช้เวลากี่วินาที การแยก Task 6 ออกมาจึงเป็นแค่การสร้างจุดสิ้นสุดให้ครบโครงสร้างมากกว่า

Task 7 : Script Task: Log Load + Write CSV

1. ลาก **Script Task** ลงใน Control Flow
2. เปลี่ยนชื่อเป็น `Log Load + Write CSV`
3. เชื่อมลูกศรจาก **Load to MySQL** → **Log Load + Write CSV**
4. Double-click ที่ **Log Load + Write CSV**
5. ตั้งค่า **ReadWriteVariables:**
   `User::LoadStart,User::ExtractSec,User::TransformSec,User::LoadSec,User::DatasetSize`

   > เพราะ Task 7 ต้องทำงาน 2 อย่างพร้อมกัน อย่างแรกคือคำนวณเวลา Load เหมือนที่ Task 3 และ Task 5 ทำ จึงต้องการ LoadStart และ LoadSec อย่างที่สองคือเขียนผลรวมทั้งหมดลงไฟล์ CSV จึงต้องดึง ExtractSec และ TransformSec ที่คำนวณไว้แล้วตั้งแต่ Task 3 และ Task 5 มาใช้ด้วย ส่วน DatasetSize ใช้เพื่อบอกว่าแถวที่กำลังเขียนลง CSV นี้เป็นผลของการรันขนาดไหน

6. สร้างโฟลเดอร์ data ใน `D:\ssis_credit_card_fraud_detection`
7. กด **Edit Script** → วางโค้ดนี้ใน `Main()` :

```
public void Main()
{
    var loadStart = (DateTime)Dts.Variables["User::LoadStar
t"].Value;
    double loadSec = (DateTime.Now - loadStart).TotalSecond
s;
    double extSec = (double)Dts.Variables["User::ExtractSe
c"].Value;
    double trfSec = (double)Dts.Variables["User::TransformS
ec"].Value;
    string size = (string)Dts.Variables["User::DatasetSiz
e"].Value;

    string logPath = @"D:\ssis_credit_card_fraud_detection
\data\timing_log.csv";


     /*
```

```
        บรรทัดนี้ตรวจสอบว่าไฟล์ timing_log.csv มีอยู่แล้วหรือยัง
        ถ้ายังไม่มีก็จะสร้างไฟล์ใหม่พร้อมเขียน Header บรรทัดแรกว่า datas
et, extract_sec,
        transform_sec, load_sec
   */
    if (!System.IO.File.Exists(logPath))
            // WriteAllText คือการเขียนทับไฟล์ทั้งหมด ถ้าใช้ตัวนี้ทุกครั้
งที่รัน Package ข้อมูลเก่าจะหายหมด เหลือแค่ผลของการรันล่าสุด
        System.IO.File.WriteAllText(logPath, "dataset,extra
ct_sec,transform_sec,load_sec\n");

    // AppendAllText คือการเพิ่มข้อมูลต่อท้ายไฟล์โดยไม่ลบของเดิมออก ท
ำให้หลังจากรันครบทั้ง 3 ขนาดไฟล์ CSV จะมีข้อมูลครบ 3 แถวคือ 10k 100k
และ 1m
    System.IO.File.AppendAllText(logPath, line);

    Dts.Variables["User::LoadSec"].Value = loadSec;
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

Save → **Build** → ปิด Script Editor → กด **OK**

🎉🎉🎉 ETL_10k สำเร็จ! ไปต่อที่ Phase 5 เลย